

Implementation of Wazuh SIEM with Suricata Integration

PROJECT REPORT

Prepared By

Nibin M

July 22, 2026

Contents

1	Introduction	2
1.1	Overview	2
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Scope	2
2	Tools and Technologies Used	3
2.1	Network Topology	3
2.2	Installing Wazuh Manager	3
2.3	Installing the Wazuh Agent	4
2.4	Installing Suricata	4
2.5	Starting the Suricata Service	4
3	Implementation	5
3.1	Step 1: Verifying the Wazuh Server IP Address	5
3.2	Step 2: Verifying the Wazuh Manager Service	5
3.3	Step 3: Verifying the Suricata Service on Kali	6
3.4	Step 4: Verifying the Wazuh Agent on Kali	7
3.5	Step 5: Configuring Suricata Log Integration	7
3.6	Step 6: Verifying Agent Connectivity on the Wazuh Dashboard	8
3.7	Step 7: Generating Test Network Traffic	9
3.8	Step 8: Monitoring Alerts in Threat Hunting	9
3.9	Step 9: Creating a Custom Wazuh Detection Rule	10
3.10	Step 10: Verifying the Custom Detection Rule	11
4	Results and Analysis	12
4.1	Observed Behavior	12
4.2	Sample Alert Fields	12
4.3	Discussion	12
5	Conclusion and Future Scope	13
5.1	Conclusion	13
5.2	Future Scope	13
	References	14

Chapter 1

Introduction

1.1 Overview

With the rapid growth of networked systems, organizations face an increasing number of cyber threats that exploit vulnerabilities in real time. A robust security monitoring infrastructure is essential to detect, log, and respond to malicious activity as it occurs. This project demonstrates the integration of **Suricata**, an open-source Intrusion Detection System (IDS), with **Wazuh**, an open-source Security Information and Event Management (SIEM) platform, to build a centralized threat detection and alerting system.

1.2 Problem Statement

Standalone IDS tools generate large volumes of alert data (in the form of logs such as `eve.json`) that are difficult to monitor manually. Without a centralized platform to collect, correlate, and visualize these alerts, security analysts cannot respond to threats efficiently. This project addresses that gap by forwarding Suricata alerts to a Wazuh Manager for centralized analysis.

1.3 Objectives

- To deploy a Wazuh Manager on an Ubuntu server for centralized log collection.
- To deploy a Wazuh Agent and Suricata IDS on a Kali Linux client machine.
- To integrate Suricata's `eve.json` alert output with the Wazuh agent configuration.
- To generate live network traffic (using Nmap scans) to trigger IDS detections.
- To create a custom Wazuh detection rule that classifies and alerts on Suricata events.
- To verify successful log flow from the agent to the manager dashboard.

1.4 Scope

This project is limited to a controlled lab environment consisting of two virtual machines connected on the same private network. It focuses on detection and alerting rather than active prevention (IPS mode) or automated response.

Chapter 2

Tools and Technologies Used

Tool / Technology	Purpose
Wazuh Manager (Ubuntu)	Centralized SIEM server for log collection, correlation, and alerting
Wazuh Agent (Kali Linux)	Lightweight agent that forwards logs from the client to the manager
Suricata	Open-source network IDS/IPS engine that inspects traffic and generates <code>eve.json</code> alerts
Nmap	Network scanning tool used to generate test traffic and trigger IDS signatures
Ubuntu Server 22.04 LTS	Host OS for the Wazuh Manager
Kali Linux	Host OS for Suricata and the Wazuh Agent

Table 2.1: Tools and technologies used in the project

2.1 Network Topology

- **Wazuh Server (Ubuntu):** 192.168.42.201
- **Kali Linux (Agent + Suricata):** 192.168.42.157

Both machines are connected on the same private/host-only virtual network to allow agent-manager communication and to permit Nmap scans to reach the target host for test-traffic generation.

2.2 Installing Wazuh Manager

The Wazuh Manager and Dashboard were installed on an Ubuntu virtual machine using the official Wazuh installation assistant. The following commands were used to download and execute the installation script.

```
curl -s0 https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

After the installation completed successfully, the Wazuh Manager and Dashboard services were started automatically. The generated administrator credentials were used to access the web dashboard.

2.3 Installing the Wazuh Agent

The Wazuh Agent was installed on the Kali Linux virtual machine and configured to communicate with the Wazuh Manager.

```
wget https://packages.wazuh.com/4.x/apt/pool/main/w/wazuh-agent/
wazuh-agent_4.14.6-1_amd64.deb

sudo WAZUH_MANAGER="192.168.42.201" \
WAZUH_AGENT_NAME="Kali" \
WAZUH_AGENT_GROUP="default" \
dpkg -i wazuh-agent_4.14.6-1_amd64.deb

sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

2.4 Installing Suricata

Suricata was installed on the Kali Linux virtual machine using the APT package manager.

```
sudo apt update

sudo apt install suricata -y
```

After installation, the Emerging Threats rule set was downloaded and updated.

```
sudo suricata-update
```

2.5 Starting the Suricata Service

The Suricata configuration was updated to monitor the active network interface. After modifying the configuration, the service was enabled and started.

```
sudo systemctl enable suricata

sudo systemctl restart suricata

sudo systemctl status suricata
```

Chapter 3

Implementation

3.1 Step 1: Verifying the Wazuh Server IP Address

The first step is to confirm the IP address and hostname of the Ubuntu machine hosting the Wazuh Manager, since the Wazuh Agent on Kali will need this address to register and communicate with the manager.

```
$ ip -br addr
```

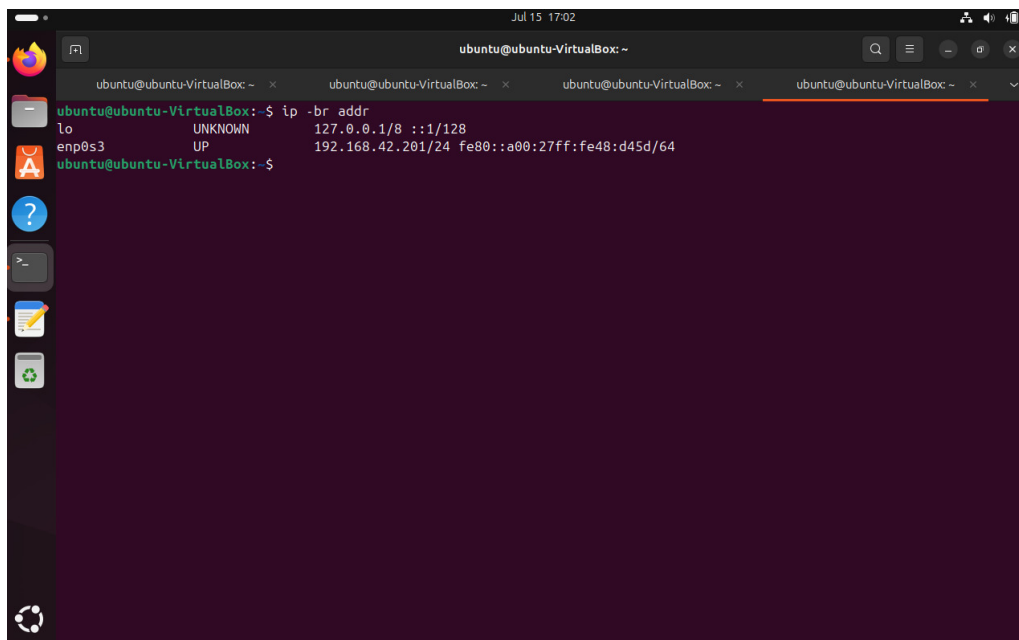
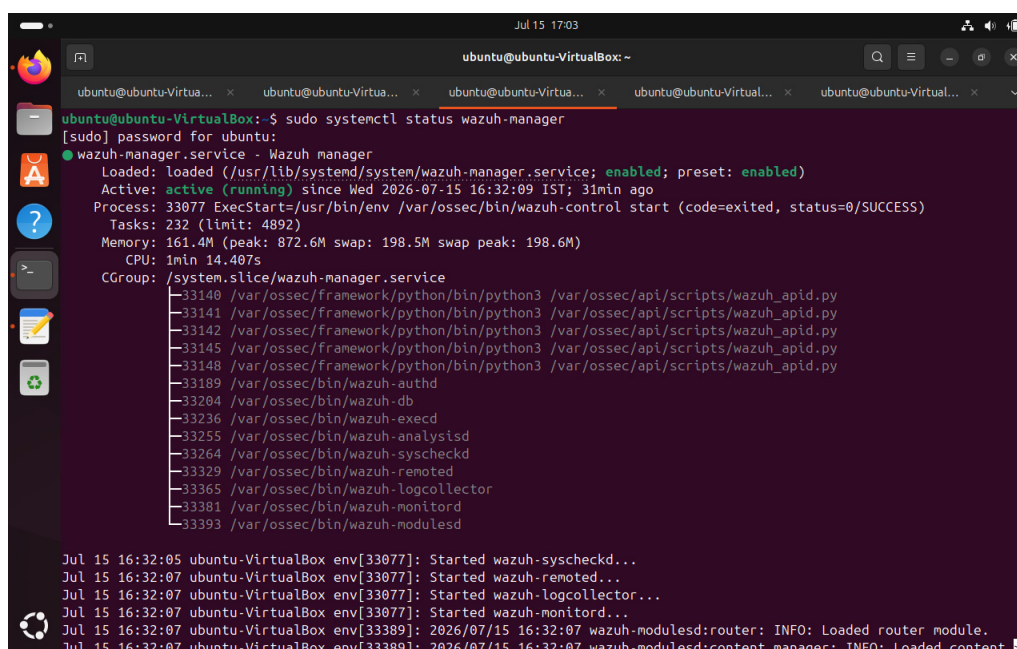


Figure 3.1: Ubuntu Wazuh Server – Network Interface and IP Address

3.2 Step 2: Verifying the Wazuh Manager Service

Once installed, the Wazuh Manager service must be active for the platform to accept agent connections and process incoming events.

```
$ sudo systemctl status wazuh-manager
```



```

Jul 15 17:03
ubuntu@ubuntu-VirtualBox: ~
ubuntu@ubuntu-VirtualBox: ~$ sudo systemctl status wazuh-manager
[sudo] password for ubuntu:
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-07-15 16:32:09 IST; 31min ago
     Process: 33077 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
    Tasks: 232 (limit: 4892)
   Memory: 161.4M (peak: 872.6M swap: 198.5M swap peak: 198.6M)
      CPU: 1min 14.407s
   CGroup: /system.slice/wazuh-manager.service
           └─33140 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             33141 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             33142 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             33145 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             33148 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
             33189 /var/ossec/bin/wazuh-authd
             33204 /var/ossec/bin/wazuh-db
             33236 /var/ossec/bin/wazuh-execd
             33255 /var/ossec/bin/wazuh-analysisd
             33264 /var/ossec/bin/wazuh-syscheckd
             33329 /var/ossec/bin/wazuh-remoted
             33365 /var/ossec/bin/wazuh-logcollector
             33381 /var/ossec/bin/wazuh-monitord
             33393 /var/ossec/bin/wazuh-modulesd

Jul 15 16:32:05 ubuntu-VirtualBox env[33077]: Started wazuh-syscheckd...
Jul 15 16:32:07 ubuntu-VirtualBox env[33077]: Started wazuh-remoted...
Jul 15 16:32:07 ubuntu-VirtualBox env[33077]: Started wazuh-logcollector...
Jul 15 16:32:07 ubuntu-VirtualBox env[33077]: Started wazuh-monitord...
Jul 15 16:32:07 ubuntu-VirtualBox env[33389]: 2026/07/15 16:32:07 wazuh-modulesd:router: INFO: Loaded router module.
Jul 15 16:32:07 ubuntu-VirtualBox env[33389]: 2026/07/15 16:32:07 wazuh-modulesd:content manager: INFO: Loaded content

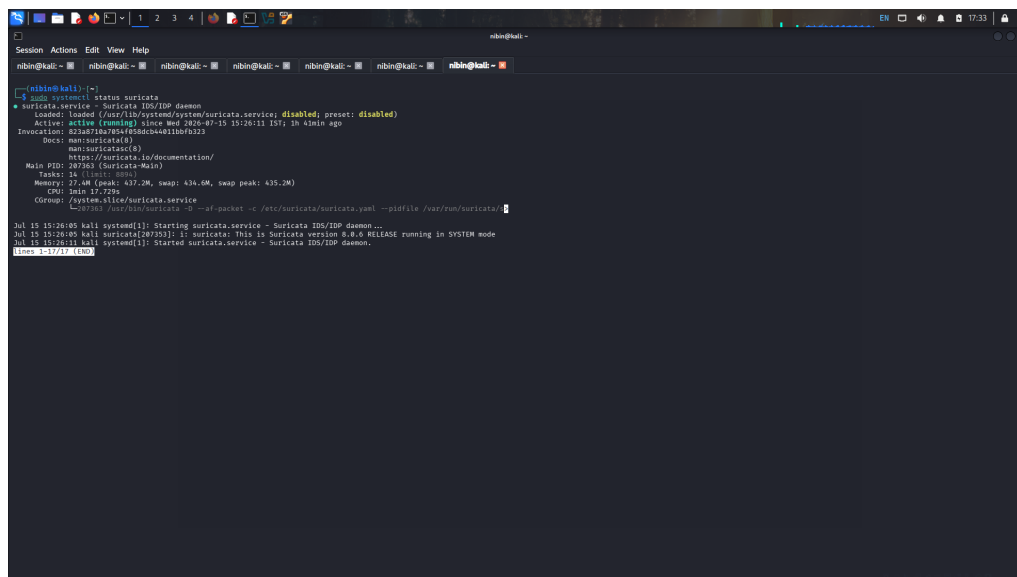
```

Figure 3.2: Wazuh Manager Service Status

3.3 Step 3: Verifying the Suricata Service on Kali

Suricata must be running on the Kali Linux client to inspect network traffic in real time and generate alert logs.

```
$ sudo systemctl status suricata
```



```

nbin@kali: ~$ sudo systemctl status suricata
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; disabled; preset: disabled)
   Active: active (running) since Wed 2026-07-15 15:20:11 IST; 1h 41min ago
     Invocation: 813ab718a765f68d8dc4ae11bfb32
       Doc: man:suricata(8)
    Main PID: 207863 (Suricata-Main)
      Tasks: 35 (limit: 65536)
   Memory: 27.4M (peak: 637.2M, swap: 634.0M, swap peak: 635.2M)
      CPU: 15 (limit: 60%)
   CGroup: /system.slice/suricata.service
           └─207863 /usr/bin/suricata -D -s /etc/suricata/suricata.yaml --pidfile /var/run/suricata/d

Jul 15 15:20:05 kali systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Jul 15 15:20:09 kali suricata[207863]: 1: suricata: This is Suricata version 0.0.0 RELEASE running in SYSTEM mode
Jul 15 15:20:11 kali systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.

```

Figure 3.3: Suricata IDS Service Status on Kali Linux

3.4 Step 4: Verifying the Wazuh Agent on Kali

The Wazuh Agent must also be active on the Kali machine so that it can forward Suri-cata's logs to the central manager.

```
$ sudo systemctl status wazuh-agent
```

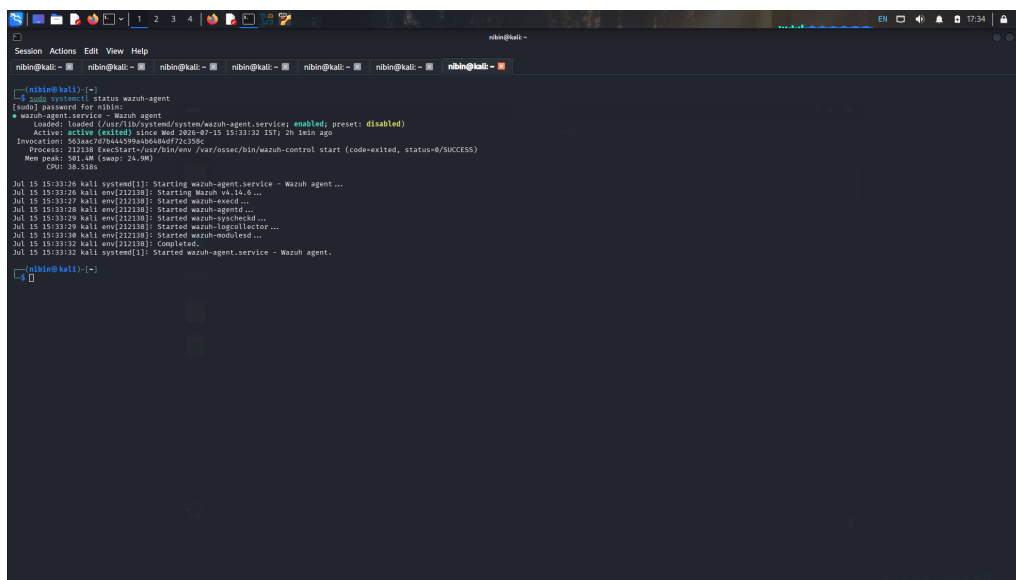


Figure 3.4: Wazuh Agent Service Status on Kali Linux

3.5 Step 5: Configuring Suricata Log Integration

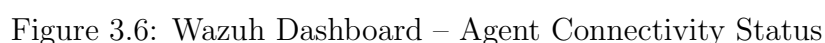
To forward Suricata's JSON alert output to Wazuh, the agent configuration file `ossec.conf` is edited to add a log-collection block pointing to Suricata's `eve.json` file.

```
$ sudo nano /var/ossec/etc/ossec.conf
```

```
1 <localfile>
2   <log_format>json</log_format>
3   <location>/var/log/suricata/eve.json</location>
4 </localfile>
```




After restarting the agent, the Wazuh web dashboard is checked to confirm that the Kali Linux agent shows an **Active** connection status to the manager.



3.7 Step 7: Generating Test Network Traffic

To validate that Suricata detects and logs activity, Nmap scans are launched against the Kali host from another machine on the network. An aggressive scan (-A) and a vulnerability scan (--script vuln) are both used to intentionally trigger Suricata's detection signatures.

```
$ nmap -A 192.168.42.157
$ nmap --script vuln 192.168.42.157
```

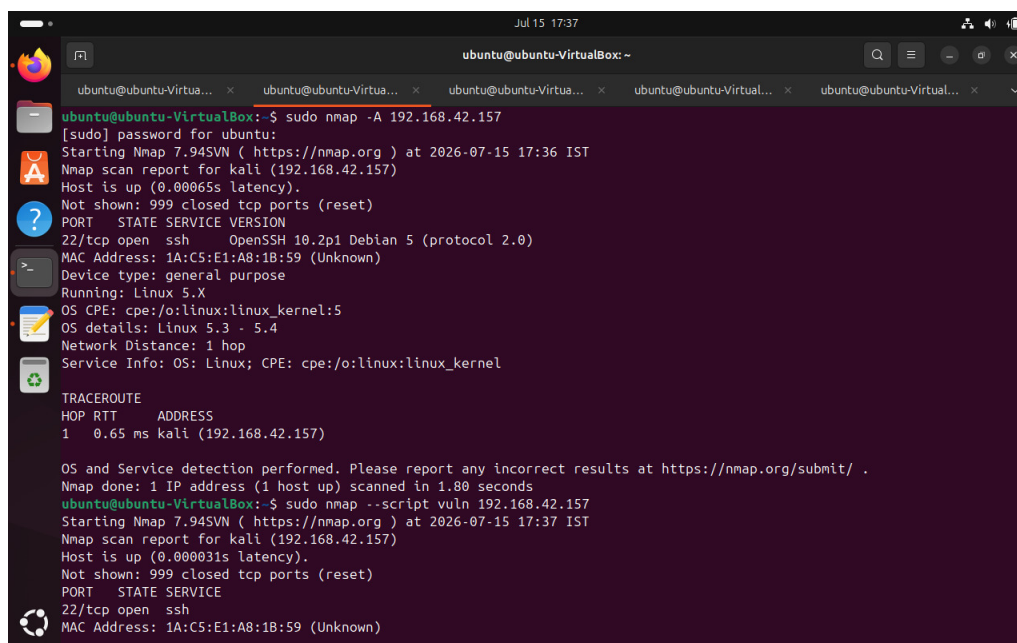


Figure 3.7: Test Traffic Generation – Nmap Aggressive and Vulnerability Scans

3.8 Step 8: Monitoring Alerts in Threat Hunting

After generating the test traffic using Nmap, the Threat Hunting module in the Wazuh Dashboard was used to verify that Suricata alerts were successfully received and processed.

The dashboard displayed information such as the alert description, severity level, source and destination IP addresses, timestamp, and the corresponding Wazuh rule ID. This confirms that the integration between Suricata and Wazuh is functioning correctly.

timestamp	agent.name	rule.description	rule.level	rule.id
Jul 15, 2026 @ 17:38:01.184	ubuntu-VirtualBox	Successful sudo to ROOT executed.	3	5402
Jul 15, 2026 @ 17:38:01.184	ubuntu-VirtualBox	PAM: Login session opened.	3	5501
Jul 15, 2026 @ 17:37:11.671	Kali	Custom Detection: By Nibi's Suricata alert escalated	12	100100
Jul 15, 2026 @ 17:37:11.158	ubuntu-VirtualBox	PAM: Login session closed.	3	5502
Jul 15, 2026 @ 17:37:01.154	ubuntu-VirtualBox	PAM: Login session opened.	3	5501
Jul 15, 2026 @ 17:37:01.154	ubuntu-VirtualBox	Successful sudo to ROOT executed.	3	5402
Jul 15, 2026 @ 17:36:57.152	ubuntu-VirtualBox	PAM: Login session closed.	3	5502
Jul 15, 2026 @ 17:36:55.665	Kali	Custom Detection: By Nibi's Suricata alert escalated	12	100100
Jul 15, 2026 @ 17:36:55.662	Kali	Custom Detection: By Nibi's Suricata alert escalated	12	100100
Jul 15, 2026 @ 17:36:55.153	ubuntu-VirtualBox	Successful sudo to ROOT executed.	3	5402
Jul 15, 2026 @ 17:36:55.153	ubuntu-VirtualBox	PAM: Login session opened.	3	5501
Jul 15, 2026 @ 17:36:01.689	Kali	PAM: Login session closed.	3	5502
Jul 15, 2026 @ 17:35:45.733	Kali	PAM: Login session opened.	3	5501
Jul 15, 2026 @ 17:35:45.691	Kali	Successful sudo to ROOT executed.	3	5402
Jul 15, 2026 @ 17:34:53.697	Kali	PAM: Login session closed.	3	5502

Figure 3.8: Wazuh Dashboard – Suricata Alerts

3.9 Step 9: Creating a Custom Wazuh Detection Rule

A custom rule is added to `local_rules.xml` so that Wazuh can classify incoming Suricata alerts under a distinct rule ID, enabling clearer identification and severity tagging in the dashboard.

```
$ sudo nano /var/ossec/etc/rules/local_rules.xml
```

```
1 <rule id="100100" level="12">
2   <if_sid>86601</if_sid>
3   <description>Custom Detection: By Nibi's Suricata alert escalated
4   </description>
5   <group>project_demo,custom_detection by Nibin.</group>
6 </rule>
```

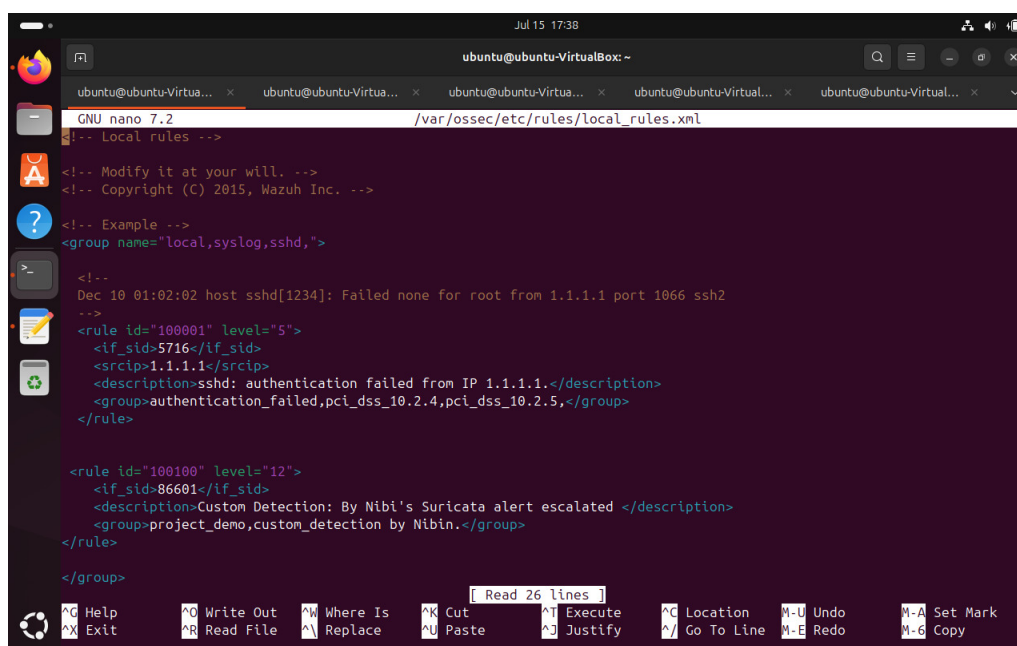


Figure 3.9: Custom Wazuh Rule Definition

3.10 Step 10: Verifying the Custom Detection Rule

To improve the detection capability of Wazuh, a custom detection rule (Rule ID 100100) was created in the `local_rules.xml` file. This rule extends Wazuh's built-in Suricata rule (Rule ID 86601) using the `<if_sid>` directive and specifically identifies Nmap scan activities.

After saving the custom rule and restarting the Wazuh Manager, the Nmap scan was executed again to generate new Suricata alerts. The Threat Hunting dashboard confirmed that the newly generated events matched the custom rule and were displayed with Rule ID 100100.

This verification confirms that the custom detection rule was successfully loaded and that Wazuh can be extended to classify specific security events with customized severity levels and descriptions.



Figure 3.10: Custom Rule ID 100100 Triggered in Threat Hunting

Chapter 4

Results and Analysis

4.1 Observed Behavior

After configuring the log integration and restarting the Wazuh agent, Suricata alerts generated during the Nmap scans were successfully parsed and forwarded to the Wazuh Manager. Alerts matching custom rule 100100 appeared on the Wazuh dashboard under the Suricata alert group, along with metadata such as source/destination IP, port, and the Suricata signature that was triggered.

4.2 Sample Alert Fields

Field	Description
<code>event_type</code>	Identifies the record as a Suricata alert event
<code>alert.signature</code>	Human-readable description of the detected pattern
<code>src_ip / dest_ip</code>	Source and destination of the flagged traffic
<code>rule.id</code>	Wazuh rule that matched (100100)
<code>rule.level</code>	Severity level assigned (10)

Table 4.1: Key fields observed in the generated alerts

4.3 Discussion

The successful triggering of custom rule 100100 during the Nmap scans confirms that the end-to-end pipeline – from packet capture by Suricata, through log forwarding by the Wazuh Agent, to rule matching on the Wazuh Manager – functions correctly. This validates the core objective of building a centralized, real-time detection pipeline using open-source tools.

Chapter 5

Conclusion and Future Scope

5.1 Conclusion

This project successfully demonstrated the integration of Suricata IDS with the Wazuh SIEM platform in a two-host lab environment. By configuring log forwarding and a custom detection rule, network reconnaissance activity generated using Nmap was detected, logged, and centrally visualized. The exercise highlights how open-source tools can be combined to build an effective and cost-efficient network security monitoring solution.

5.2 Future Scope

- Enabling Suricata in **IPS (inline) mode** to actively block malicious traffic rather than only detecting it.
- Integrating additional log sources (e.g., firewall, Windows Event Logs, web server logs) into Wazuh for broader visibility.
- Configuring automated active-response scripts in Wazuh to react to high-severity alerts.
- Deploying the setup across a larger, multi-agent network to evaluate scalability.
- Integrating threat-intelligence feeds to enrich Suricata's rule set with up-to-date IOCs.

References

1. Wazuh Documentation – <https://documentation.wazuh.com>
2. Suricata Documentation – <https://suricata.readthedocs.io>
3. Nmap Reference Guide – <https://nmap.org/book/man.html>